

1.

a) (1 ponto)

Analise o código seguinte:

```
unsigned int id;
string description;
float price;
cout << "ID ? "; cin >> id;
cout << "DESCRIPTION ? "; getline(cin, description);
cout << "PRICE ? "; cin >> price;
```

Apresenta-se a seguir a interação com um utilizador, ao ser executado este código (o número sublinhado, 123, representa a entrada do utilizador ao ser executada a instrução `cin >> id`); note que ao executar `getline(cin, description)` o programa não esperou pela entrada do utilizador.

ID ? 123

DESCRIPTION ? PRICE ? _ (cursor a piscar)

Explique, brevemente, o que aconteceu e indique uma solução para o problema.

O que aconteceu: ficou um carácter *newline* no *buffer* de entrada, após a leitura de "id"; esse carácter foi lido por `getline()` que retornou imediatamente, sem esperar pela entrada do utilizador

Solução: eliminar o *newline* que ficou no *buffer* com a instrução `cin.ignore(1000, '\n')` ou `cin.ignore(numeric_limits<streamsize>::max(), '\n')`;

b) (1 ponto)

Um programador que desenvolveu um programa que incluía o código abaixo ficou surpreendido com os resultados pois obteve as saídas:

```
result = 2
result = 0
```

Identifique o problema e indique as correções necessárias para que os resultados sejam corretos.

```
double calculate(int x, char op, int y)
{
    double result;
    switch (op)
    {
        case '+':
            result = x + y;
        case '-':
            result = x - y;
        case '*':
            result = x * y;
        case '/':
            result = x / y;
    }
    return result;
}

int main()
{
    std::cout << "result = " << calculate(5, '*', 2) << std::endl;
    std::cout << "result = " << calculate(2, '+', 3) << std::endl;
    ...
}
```

O problema: faltam "break"'s em cada "case"; por isso, o resultado é sempre o da divisão inteira

Solução (fazer 2 coisas):

- inserir "break;" após cada atribuição "result = ..."
- para que o resultado da divisão seja o esperado, substituir `result=x/y`; por `result=(double)x/y`;

c) (1 ponto)

Complete o código seguinte para que o ciclo seja executado até que o valor introduzido pelo utilizador seja um Y ou N, em maiúsculas ou minúsculas.

```
char value;
do {
    cout << "Continue (Y/N) ? ";
    cin >> value;
} while (...); // completar esta linha de código
```

Várias soluções alternativas:

- `while ( ! (value=='Y' || value=='y' || value=='N' || value=='n'));`
- `while ( value!='Y' && value!='y' && value!='N' && value!='n');`
- converter `value` p/maiúsculas usando `toupper()` e testar apenas os valores 'Y' e 'N' ou p/minúsculas, usando `tolower()` e testar apenas os valores 'y' e 'n'

d) (1 ponto)

A função `void concatenateIntVectors(...)` tem como parâmetros 2 vetores de inteiros, `v1` e `v2`, e modifica o vetor `v1`, acrescentando no final deste vetor os elementos do vetor `v2`. Complete o protótipo e escreva o código da função.

```
void concatenateIntVectors(vector<int> &v1, const vector<int> &v2) //notar & e const
{
    for (size_t i=0; i<v2.size(); i++)
        v1.push_back(v2.at(i));
}
```

e) (1 ponto)

Considere a instrução seguinte de um programa em que `s1` e `s2` estão declaradas como sendo variáveis do tipo `string`:

```
cout << s1 + s2;
```

Qual o resultado desta instrução nas seguintes duas situações:

A) `s1` toma o valor `"123"` e `s2` toma o valor `"12"`

B) `s1` toma o valor `"123"` e `s2` toma o valor `"\n12"`.

A) Aparece no ecrã:

12312

B) Aparece no ecrã:

123

12

f) (1 ponto)

Qual a saída do extrato de código seguinte?

```
int a[10];
for (int i = 0; i < 10; i++)
    a[i] = 10 * (i + 1);
```

```
int *p = a;
```

```
cout << *(p + 2) << endl;
```

A saída é:

30

2. (7 pontos)

Um ficheiro de texto contém um conjunto de números inteiros, na gama [0..20], um número em cada linha. Pretende-se desenvolver um programa que leia os números e calcule o histograma dos valores lidos. Recorda-se que um histograma de um conjunto de valores indica o número de ocorrências de cada um dos valores (no exemplo apresentado abaixo, o valor 0 ocorre 1 vez, o valor 1 ocorre 1 vez, ..., o valor 19 ocorre 2 vezes e o valor 20 ocorre 0 vezes).

O programa deve apresentar no ecrã todos os números lidos e o seu histograma. Apresenta-se a seguir um exemplo do conteúdo de um ficheiro ("numbers.txt") e da interação do programa com o utilizador, ao ser executado. Os resultados devem ser apresentados com a formatação ilustrada no exemplo.

Escreva o código do programa. Se o ficheiro indicado pelo utilizador não existir, o programa deve apresentar a mensagem "File not found!" e terminar. Se o ficheiro existir, considere que tem um conteúdo válido, isto é, apenas texto que representa números inteiros na gama indicada anteriormente.

EXEMPLO:

Conteúdo do ficheiro numbers.txt:

```
19
10
14
1
13
11
19
14
14
17
13
0
10
```

Resultado de execução do programa:

```
Filename? numbers.txt

Values
-----
19
10
.
. // outros valores
.
13
0
10

Histogram
-----
0 - 1
1 - 1
2 - 0
.
. // outros valores
.
18 - 0
19 - 2
20 - 0
```

```
#include <iostream>
#include <vector>
#include <iomanip>

using namespace std;

int main()
{
    // Read filename
    string filename;
    cout << "Filename ? ";
    getline(cin, filename);

    // Try to open filename
    ifstream f(filename);
    if (!f.is_open())
    {
        cout << "File not found!\n";
        return 1;
    }

    // Initialize histogram
    vector<int> hist(21, 0);

    // Read numbers, show them & obtain histogram
    cout << "\nValues\n-----\n";
    size_t count = 0;
    int value;
    while (f >> value)
    {
        cout << value << endl;
        hist[value]++;
        count++;
    }
    f.close();
    cout << endl;
    cout << "num values = " << count << endl;

    // Show histogram
    cout << "\nHistogram\n-----\n";
    for (size_t i = 0; i < hist.size(); i++)
        cout << setw(2) << i << " - " << setw(2) << hist[i] << endl;
    cout << endl;

    return 0;
}
```

3. (7 pontos)

Relembre o 1º projeto proposto nesta unidade curricular (Supermercado "Vende++"), onde foi desenvolvida uma aplicação que regista os produtos comprados pelos clientes de um supermercado e faz recomendações de produtos a comprar por um dado cliente, tendo em conta as compras feitas por clientes "semelhantes". Para desenvolver esse projeto, um programador definiu, entre outras, as seguintes estruturas de dados:

```
vector < string > clients; // contém os nomes dos clientes
vector < string > products; // contém os nomes dos produtos
vector < vector < bool > > transactions; // regista as compras efetuadas pelos clientes;
// transactions[i][j] tomará o valor
// true se o cliente i comprou o produto j ou
// false se o cliente i não comprou o produto j
```

Escreva o código de uma função `void listClientProducts(const vector <string> &clients, const vector <string> &products, const vector < vector < bool > > &transactions)` que mostra no ecrã o nome de cada cliente e a lista de produtos que ele comprou, como ilustrado no exemplo seguinte.

EXEMPLO:

```
Ana Sousa: arroz, chocolate, laranja
Pedro Silva: chocolate, sumo
...
```

```
void listClientProducts(const vector<string> &clients,
                       const vector<string> &products,
                       const vector<vector<bool>> &transactions)
{
    for (size_t i = 0; i < transactions.size(); i++)
    {
        cout << clients[i];
        unsigned int numProducts = 0; // alternativa: usar uma var. booleana p/indicar se já foi escrito o 1º produto
        for (size_t j = 0; j < transactions[i].size(); j++)
        {
            if (transactions[i][j] == true)
            {
                if (numProducts == 0)
                    cout << ": " << products[j];
                else
                    cout << ", " << products[j];
                numProducts++;
            }
        }
        cout << endl;
    }
}
```

FIM